

Recursion on Trees

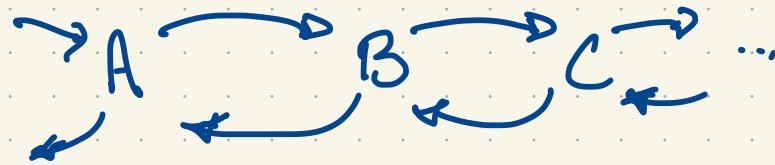
CMP-225

SPRING 2025



The call stack

- Suppose a function A calls another function B, which calls C.
- During execution, control passes from (the code for) A, to B, then to C.
- When execution of C ends, control must return to B, and then to A:



- At each function call, the system records where control should return to by pushing an activation record on the call stack.
- The call stack also records all local variables, including the arguments to the function call.

Call Stack Illustration

```
int main() {  
    int a = 1;  
    int b = f(2);  
    return 0;  
}
```

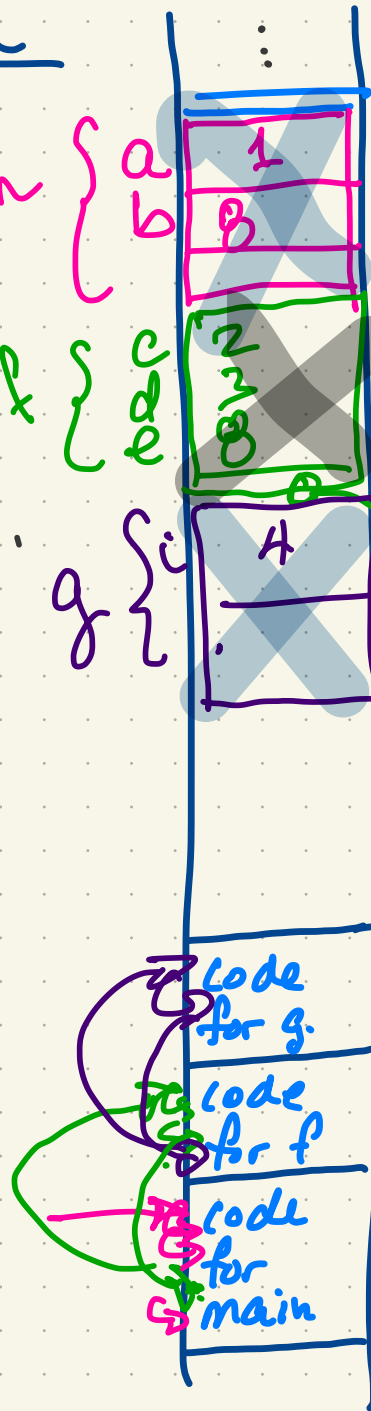
```
int f(int c) {  
    int d = 3;  
    int e = g(4);  
    return e;  
}
```

```
int g(int i) {  
    return 2 * i;  
}
```

main

f

g



call stack for this process

Recursion: A definition of a function is recursive if the body contains an application of itself.

Consider: $S(n) = \sum_{i=0}^n i$

or:
$$S(n) = \begin{cases} 0 & \text{if } n=0 \\ n + S(n-1) & \text{if } n > 0 \end{cases}$$

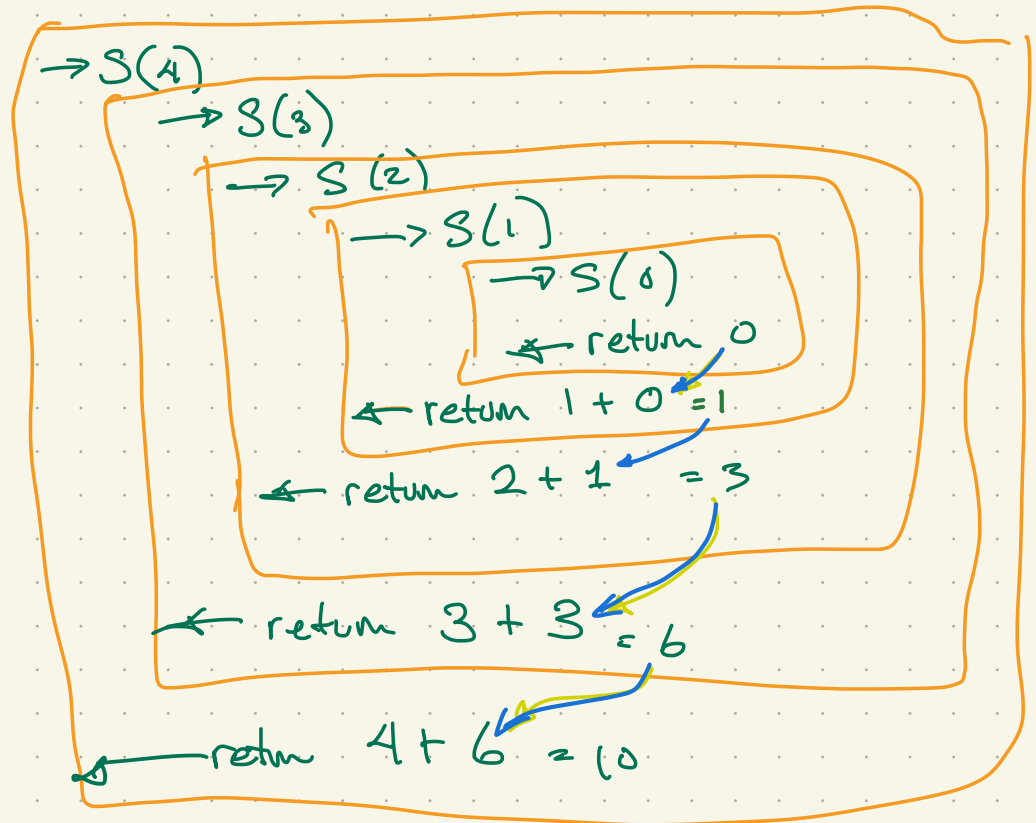
These two descriptions of $S(n)$ suggest two implementations:

→
$$S(n) \{ \begin{array}{l} s = 0 \\ \text{for } i = 1 \dots n \\ \quad s = s + i \\ \text{return } s \end{array} \}$$

$$S(n) \{ \begin{array}{l} \text{if } n = 0 \\ \quad \text{return } 0 \\ \text{else} \\ \quad \text{return } n + \underline{S(n-1)} \end{array} \}$$

Recursive Version:

```
S(n){  
  if n=0  
    return 0  
  else  
    return n + S(n-1)  
}
```



Iterative Version: $S = 0 + \sum_{i=1}^n i = 1 + \sum_{i=2}^n i$

```
S(n){  
  s = 0  
  for i = 1..n  
    s = s + i  
  return s  
}
```

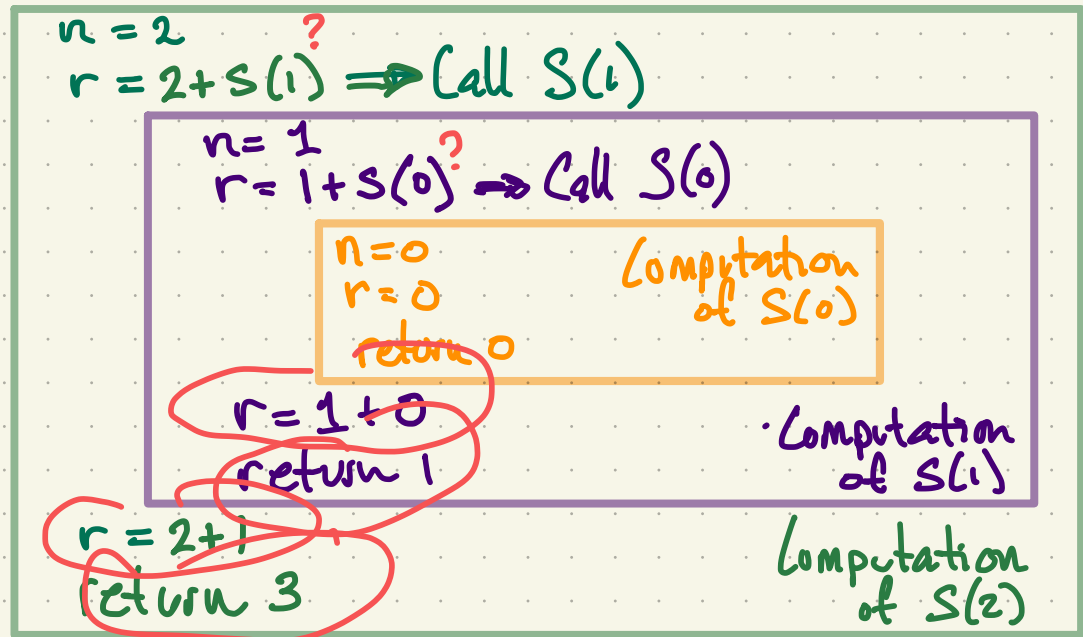
$$\begin{aligned} &= 3 + \sum_{i=3}^n i \\ &= 6 + \sum_{i=4}^n i \\ &\vdots \end{aligned}$$

• The same computation, but a different control strategy.

Recursion & The Call Stack

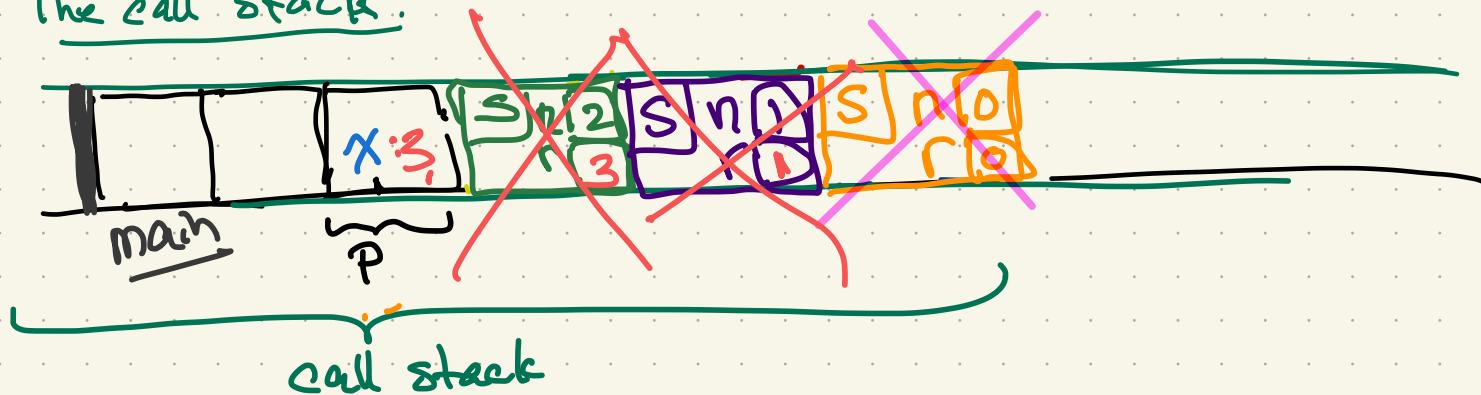
```
S(n){  
  if(n=0){  
    r=0  
  }else{  
    r=n+S(n-1)  
  }  
  return r  
}  
PO{  
  x = S(2)  
}
```

$x = S(2) \Rightarrow \text{Call } S(2)$



The call stack:

$x = 3$



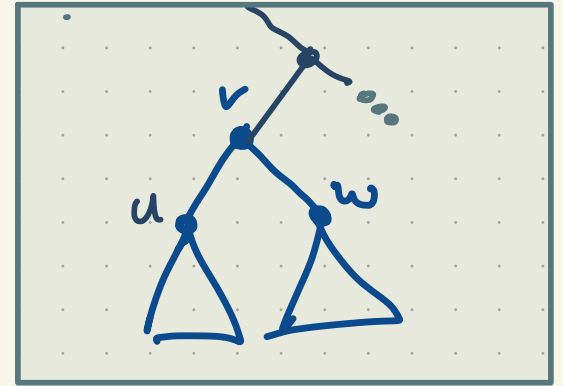
Recursion on Trees

- We will often use recursion & induction on trees.
 - eg. the tree rooted at v has some property if its subtrees have some (related) property.

- Eg: The height of a node v in a binary tree may be defined by:

$$h(v) = \begin{cases} 0 & \text{if } v \text{ is a leaf} \\ 1 + \max \{ h(\text{left}(v)), h(\text{right}(v)) \} & \text{o.w.} \end{cases}$$

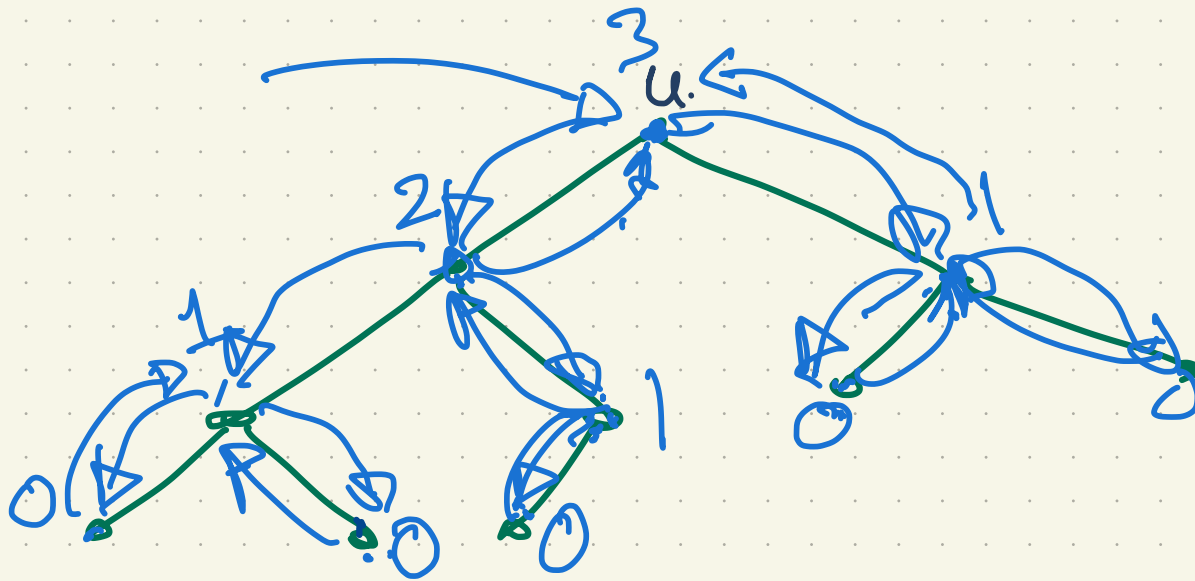
Provided we define $h(\text{left}(v))$ to be -1 if $\text{left}(v)$ does not exist, and sim. for $\text{right}(v)$.



Recursion on Trees Example

height of node v in T :

$$h(v) = \begin{cases} 0 & \text{if } v \text{ is a leaf} \\ 1 + \max\{h(\text{left}(v)), h(\text{right}(v))\}, & \text{otherwise.} \end{cases}$$



$h(u) = ?$

Pseudo-code version

height(v) {

 if v is a leaf
 return 0

 if v has one child u
 return $1 + \text{height}(u)$

 else

 return $1 + \max(\text{height}(\text{left}(v)),$
 $\text{height}(\text{right}(v)))$

}

Traversals of Binary Trees

- A traversal of a graph is a process that "visits" each node in the graph once.

- We consider 4 standard tree traversals:

1. level order (level order later).

2. pre-order

3. in-order

4. post-order.

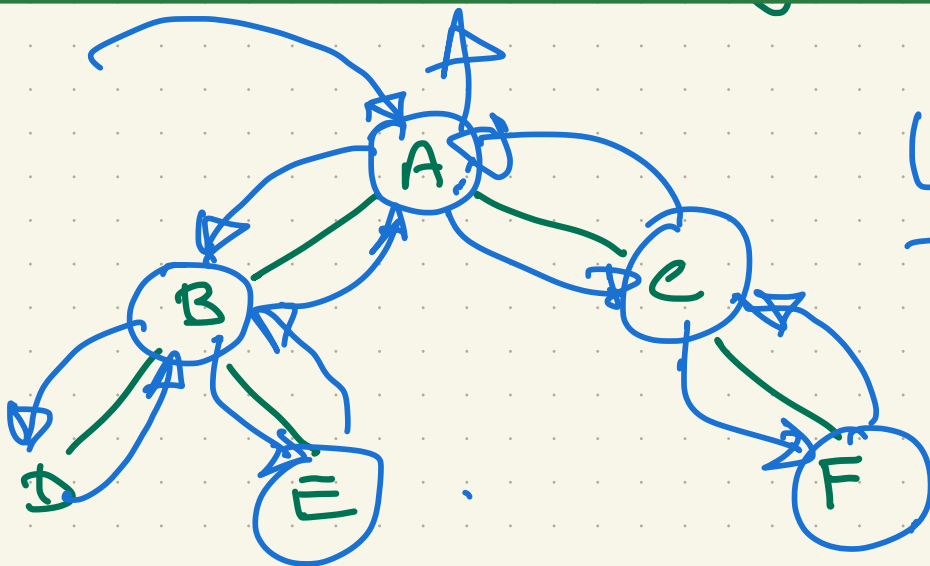
- 2,3,4 begin at the root & recursively visit the nodes in each subtree & the root. They vary in the relative order.

Pre-order Traversal of Binary Trees.

```
pre-order-T(v) {  
    visit v  
    → pre-order-T(left(v))  
    → pre-order-T(right(v))  
}
```

Notice missing base cases!
pre-order-T(v) does nothing
if v does not exist.

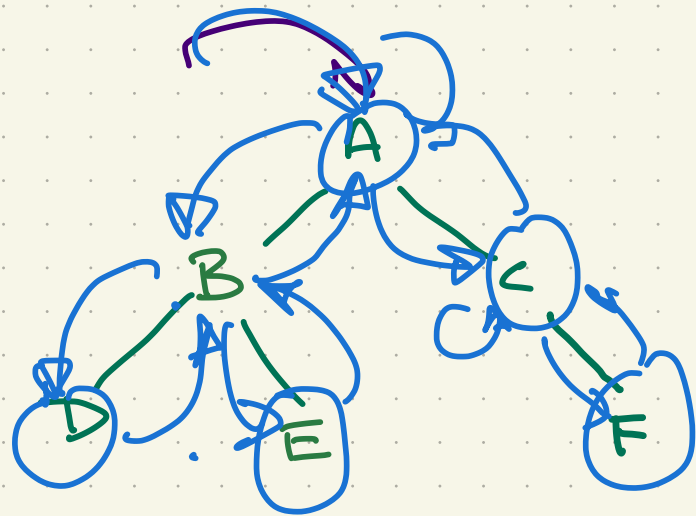
- v is visited before any of its descendants
- every node in the left subtree is visited before any node in the right subtree.



A, B, D, E, C, F

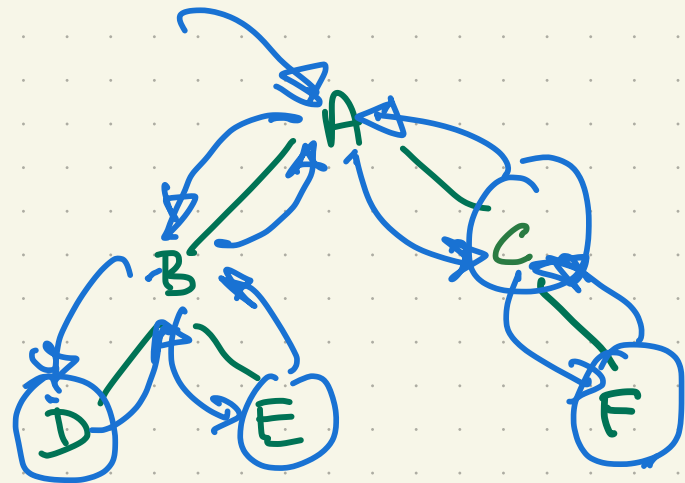
In- and Post-Order Traversals of Binary Trees

in-order-T(v) {
 ~~in-order-T(left(v))~~
 - visit v
 in-order-T(right(v))
}



D, B, E, A, C, F

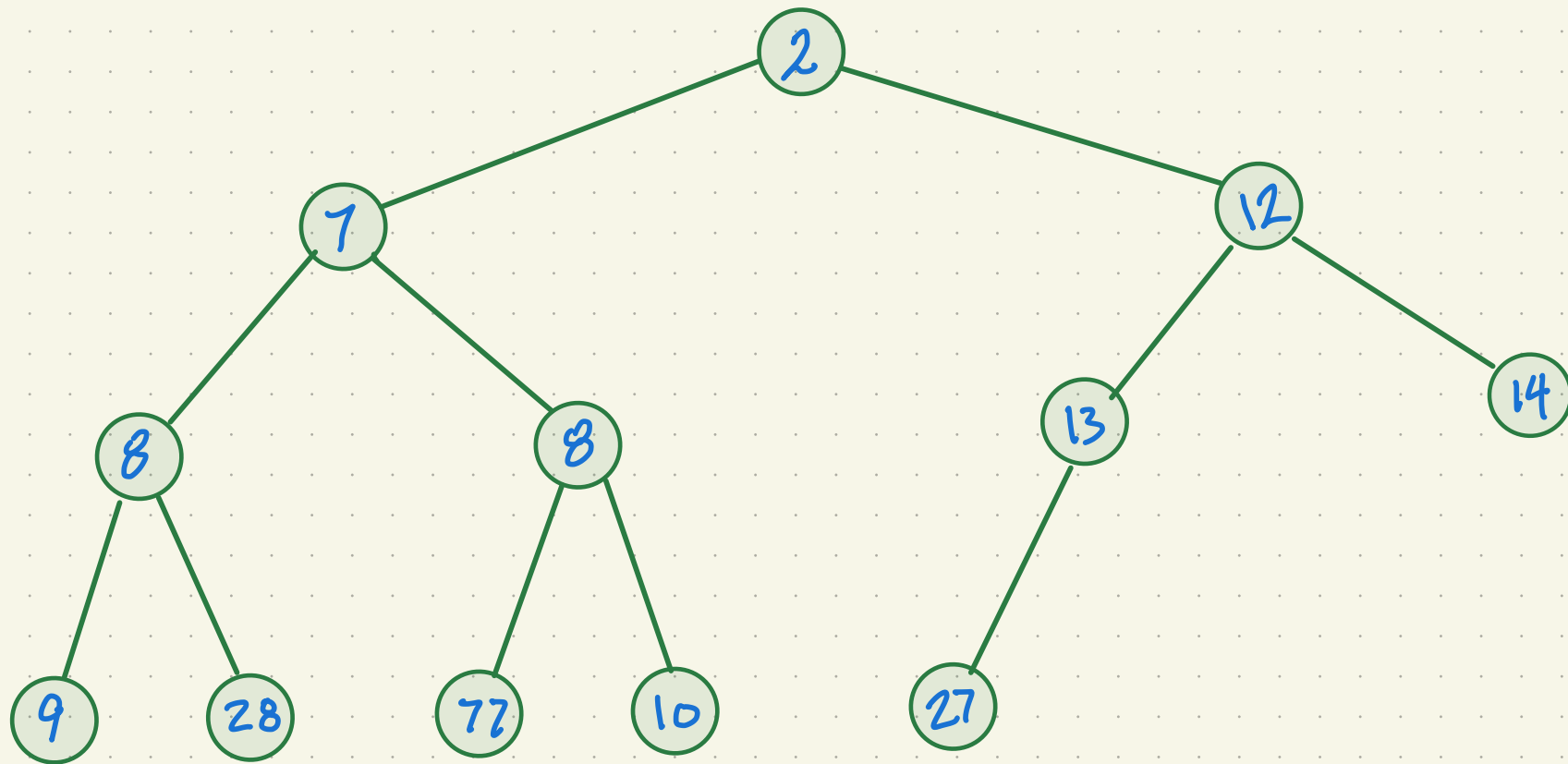
post-order-T(v) {
 ~~post-order-T(left(v))~~
 ~~post-order-T(right(v))~~
 - visit v
}



D, E, B, F, C, A

Recursion on Heaps

- Notice that every sub-tree of a heap is a heap:

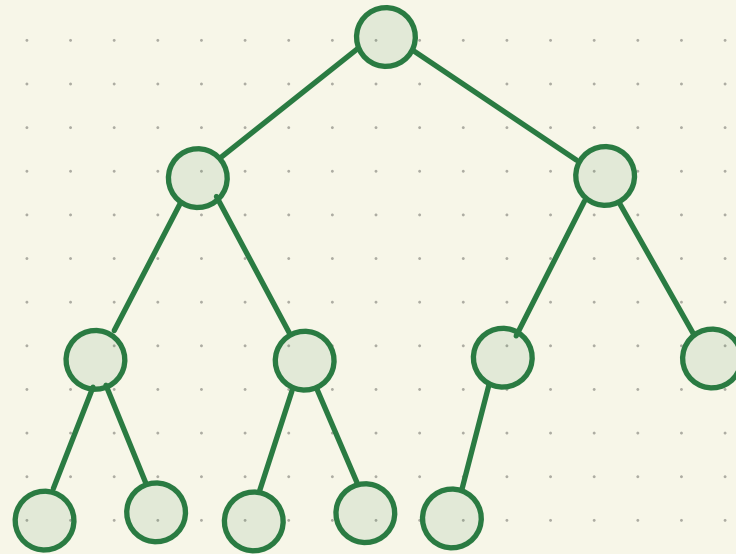
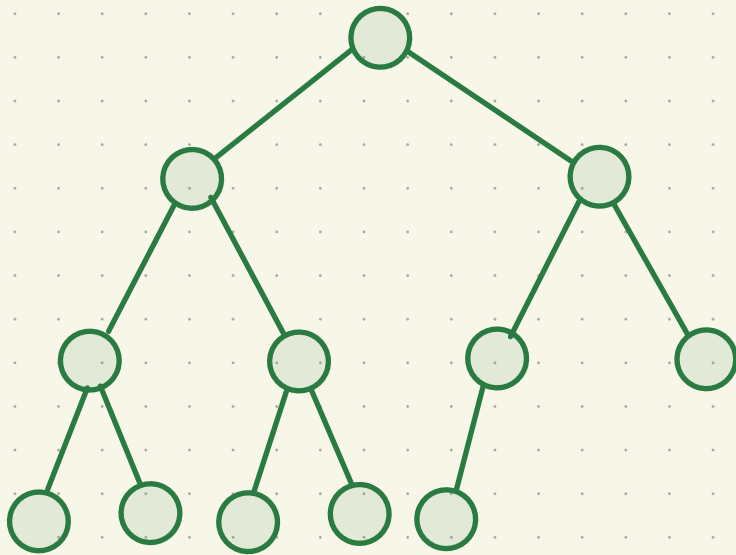


- This kind of structure makes recursive procedures natural.

Making a Heap from a Set

```
make_heap(T) {  
    // T is a complete b.t. with n keys.  
    for ( i =  $\lfloor n/2 \rfloor$  down to 1 ) {  
        call percolate-down on node i  
    }  
}
```

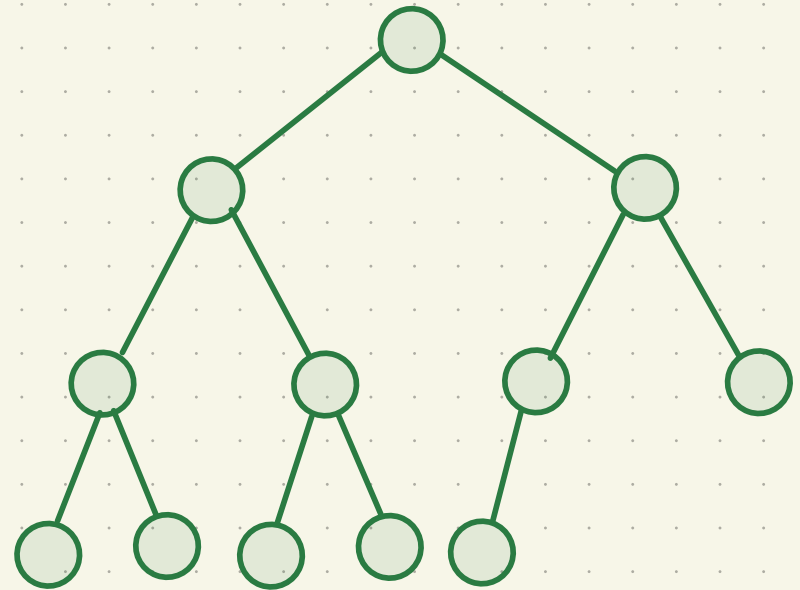
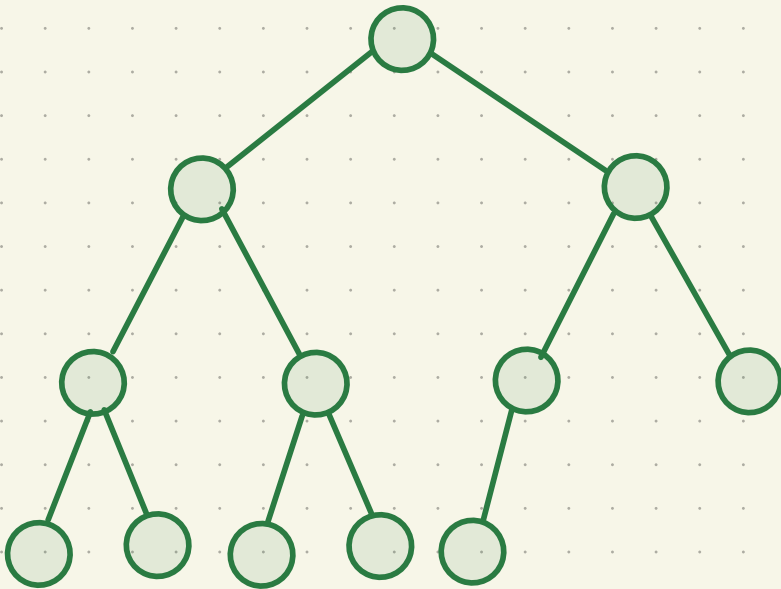
- does a percolate-down at each internal node, working bottom-up:
- at each node, makes sub-tree rooted at that node into a heap.



- The exact order of visiting nodes does not matter, provided we visit children before their parents.

Recursive Make-heap

```
make-heap(v) {  
    if (v is not a leaf) {  
        make-heap(left(v))  
        if (v has a left child) make-heap(right(v))  
        percolate-down(v)  
    }  
}
```



Fact: Any computation that can be performed as a loop can be performed recursively, and vice versa.

Example: Percolate-Up for Array-based Heap

```
insert(k){  
    size  $\leftarrow$  size + 1;  
    A[size]  $\leftarrow$  k; // Add k as the new 'last leaf'  
    v  $\leftarrow$  size  
    p  $\leftarrow$   $\lfloor v/2 \rfloor$  // p  $\leftarrow$  parent(v)  
    while (v > 1 and A[v] < A[p]){  
        swap A[v] and A[p]  
        v  $\leftarrow$  p  
        p  $\leftarrow$   $\lfloor v/2 \rfloor$   
    }  
}
```

} percolate-up

Example: Percolate-Up for Array-based Heap

```
insert(k){  
    size  $\leftarrow$  size + 1;  
    A[size]  $\leftarrow$  k;  
    percolate-up(size)  
}
```

```
percolate-up(v){  
    p  $\leftarrow$   $\lfloor v/2 \rfloor$  // p  $\leftarrow$  parent(v)  
    while (v > 1 and A[v] < A[p])  
        swap A[v] and A[p]  
        v  $\leftarrow$  p  
        p  $\leftarrow$   $\lfloor v/2 \rfloor$   
    }  
}
```

Iterative
percolate-up.

Example: Percolate-Up for Array-based Heap

```
insert(k){  
    size  $\leftarrow$  size + 1;  
    A[size]  $\leftarrow$  k;  
    percolate-up(size)  
}
```

```
percolate-up(v){  
    p  $\leftarrow$   $\lfloor v/2 \rfloor$   
    if (v > 1 AND A[v] < A[p]){  
        swap A[v] and A[p]  
        percolate-up(p)  
    }  
}
```

Recursive
percolate-up

Example: Recursive Percolate-down for Array-based heap.

```
extract_min() {  
    temp ← A[1] // record value to return  
    A[1] ← A[size] // move old last leaf to root  
    size ← size - 1  
    percolate-down(1)  
    return temp.  
}
```

```
percolate-down(i) { // Iterative Percolate-down.  
    while ( 2i ≤ size ) { // while i not a leaf  
        child ← 2i // the left child of i  
        if ( 2i+1 ≤ size AND A[2i+1] < A[2i] ) {  
            child ← 2i+1 // use the right child if it exists  
                        // and has smaller key  
        }  
        if ( A[child] < A[i] ) { // if order violated,  
            swap A[child] and A[i] // swap parent + child.  
            i ← child  
        } else {  
            return temp  
        }  
    }  
}
```

Example: Recursive Percolate-down for Array-based heap.

```
extract_min() {  
    temp ← A[1] // record value to return  
    A[1] ← A[size] // move old last leaf to root  
    size ← size - 1  
    percolate-down(1)  
    return temp.  
}
```

```
percolate-down(i) { Recursive Percolate-down.  
    if (2i ≤ size) { // i not a leaf  
        child ← 2i // the left child of i  
        if (2i+1 ≤ size AND A[2i+1] < A[2i]) {  
            child ← 2i+1 // use the right child if it exists  
                        // and has smaller key  
        }  
        if (A[child] < A[i]) { // if order violated,  
            swap A[child] and A[i] // swap parent + child.  
            percolate-down(child)  
        }  
    }  
}
```

End